SCORE: 39.0/51

Questions

1. **(2.0)** Draw a diagram showing the three possible process states and allowable transitions. Label transitions with the events that cause them. Describe how these events are caused by particular kernel operations. Diagram does not correspond exactly with that in the notes. For example, two arrows originate in nowhere.

1. INIT state: Temporary state a new process (or thread within process) enters upon creation (thread_create())

-When thread_create() called by kernel, OS makes a new Process Control Block/Thread Control Block (PCB/TCB)

-Once these are initilized, we can move these process/threads into the READY states (by adding the TCB -> READYLIST) The parent process uses the RESUME command to do this.

2. READY state

-A process is in READY state when it has all necessary resources to be a running proc but isn't in the CPU yet. It waits on the ready list (queue data struct).

-LOADSW -> RUNNING

-The OS dispatcher (kernel function) constantly monitors the ready list and when CPU is available for tasking (all this is handled by the time-out interrupt), the LOADSW op takes place saving the registers and PC from the incoming process's PCB/TCB into the CPU registers thus making it RUNNING proc.

3. RUNNING state

-process is RUNNING if currently executing instructions on the CPU. It's register values are on the CPU not the PCB/TCB.

-Within this state, the kernel op of SAVESW -> READY occurs, where:

-Time Slice is introduced, in order to ammortize process time on the CPU (using a hardware timer). The timer interrupt represents the timer being up and triggers the SAVESW op to occur taking the context of the CPU and putting it back into the appropriate PCB/TCB and therefore back into the ready list. It's two-way. Timeout --> interrupt, handler does both SAVESW and LOADSW.

SAVESW (WAIT(s) system call) -> WAITING

-In the opposite manner, if a running proc needs to wait for an external event to occur like a synchronization action (usually aiming to provide solutioning

to race conditions), the kernel makes a SAVESW cll and does the same proces of copying CPU state into requisite PCB/TCB into waiting list data struct. WAIT causes the transition, not SAVESW.

4. WAITING state

-Procs can't run in the WAITING state because they are dependent of an external action from occuring to allow them to join the ready list. The PCB/TCBs of these procs are on the waiting list.

-Kernel operation (SIGNAL(s) -> READY state):

-When event the proc waiting occurs, a different proc B can call SIGNAL(s) over the sync mechanism (ideally semaphore!), allowing the TCB/PCB to be put back into the ready list so it can become an eligible running process once again.

2. **(1.5)** The ready list can be implemented as a one-way linked list chaining TCBs (thread control blocks) together. The list itself is represented by a descriptor **`RL=(head, tail)`**, where **`RL.head`** is the index of the process at the head and **`RL.tail`** is the index of the process at the tail. A TCB has a link field (**`TLB[i].link)`** pointing to the next process after it in the queue. Draw a diagram of a ready list containing three processes waiting while a fourth is running on the CPU. With a before-after diagram, show what changes a context switch makes to this configuration. Your diagrams don't show the linked lists and how they change.

   The ready list is a one-way linked list of TCBs with a descriptor RL = (head, tail). Each TCB contains a link to the next process in the queue. When a context switch occurs, the running process's TCB is moved from CPU execution back into the ready list, and the next process at RL.head is loaded to run. This changes RL.head to point to the next ready TCB.

3. **(1.5)** Suppose that we want 2 or more CPUs to do context switching off a common ready list. Show that a race condition is possible and give an example of a system error that might result. How can this be prevented? Come up with an example.

   Since the ready list is a shared resource, that inherently creates potentiality for a race condition if implemented without proper controls. If two processes running on separate CPUs attempt to conduct a context switch simultaneously without synchronization, there exists a potential interlevement of actions that will create unexpected action.

   Multiple CPUs running the same thread: Both CPU 1 and CPU 2 read RL.head simultaneously. They both fetch Process 1 to run, believing it is the next available process. As a result, Process 1 could end up running on two CPUs concurrently. This creates system errors like memory corruption since the same process is writing to memory from multiple locations, inconsistent program state due to concurrent execution of the same instruction stream, and potential lost processes when RL.head updates interfere with each other.

What about simultaneous manipulation of RL.tail?

To prevent race conditions, we need to build and implement a spinlock. In the Test and Set Lock instruction, you read and return the contents of a memory location without any other interrupts interrupting the instruction cycle. The memory location is locked when it contains a 1, indicating that another CPU is currently accessing the critical section. (What is the critical section?) The memory location is unlocked when it contains a 0, indicating that the resource is available for use. By using the spinlock, only one CPU can enter the critical section at a time, ensuring that the operations on the ready list are atomic and that data corruption and logical errors (like two processes running on one CPU or a lost process) are prevented. The other CPU will busy-wait until the lock is released, then it can attempt to acquire it and proceed.

4. **(2.0)** The mutual exclusion locking protocol uses a semaphore **`s`** with initial counter value **`I(s)=1`**. (Note the important distinction between the semaphore counter **`s.c.`** and the initial value of that counter **`I(s)`**). The purpose of the "mutex locking protocol" is to lock a critical section of code, **C**, so that at most one process at a time can execute it. **C** is a block of instructions that manipulate shared data. In each process using the critical section **C**, the protocol is:

```
WAIT(s)
C
SIGNAL(s)
```

Prove that this protocol guarantees mutual exclusion – that is, at most one process can be executing **C** at a time. Hint: assume to the contrary that two processes are in **`C;`** from the definitions of semaphore operations work backward to conclude the only way two processes can be in **C** is a failure of the semaphore**.**

1. Assume that two processes, P1 and P2, are in the critical section at the same time.
2. In order to enter C, a single process must execute WAIT(S). When P1 executes WAIT(S), the counter changes from 1 (initial value) to 0 (decrements) and is allowed to proceed into C. The same scenario applies if P2 enters the C before P1, where WAIT(s) is decremented to 0 from initial value 1.
3. For P1/P2 to be in C at the same time, it must have also completed WAIT(S) with P1/P2 still inside. However, the counter after P1/P2 entered was decremented down to 0. If P1/P2 were to execute WAIT(S), the counter would be decremented to -1. By definition, WAIT(S) is designed to suspend the caller while the counter is negative. Meaning that P1/P2 would be blocked and unable to enter C.
4. Contradiction -> Both P1 and P2 are unable to be in C concurrently. When P1 exits and runs SIGNAL(S), the counter will increment (either from 0 to 1, -1 to 0, etc.) allowing the next process on the ready list to enter C.

a. Therefore, the protocol guarantees mutual exclusion given the semaphore implementation. PROOF FOUND.

Previous is right idea but does not exploit the contraction. All you need is:

P1 and P2 in CS -> both passed WAIT(s) -> both decremented counter -> counter = -1. But counter=-1 means one process is waiting and not in the CS as assumed.

5. **(1.0)** A strategy for avoiding deadlock with multiple locks, for example, multiple records in a bank database, is to order the locks -- say ***1, 2, ..., k, ...*** -- and impose the rule that if a process requests a lock while already holding other locks, the number of the new lock must be higher than the numbers of the locks it already holds. Show that this protocol, known as *ordered locking*, cannot deadlock. Hint: suppose that *N* processes deadlock (a circular wait) under this protocol and find a contradiction.

   1. Assume that the ordered locking protocol can result in a deadlock. No, this is not the proper setup. Assume there is a deadlock AND the ordered protocol is properly implemented.

      Assumption 1 implies resource graph has a cycle. Graph can be drawn to show the cycle, with PN requesting L1. But L1<LN contracts the proper implementation of the ordering priorities.

   2. For a deadlock to occur, there must be a circular wait among N processes. This means:
      a. Process 1 holds lock L1 and waits for lock L2 (where L1 < L2 by ordering rule)
      b. Process 2 holds lock L2 and waits for lock L3 (where L2 < L3 by ordering rule)
      c. Process 3 holds lock L3 and waits for lock L4 (where L3 < L4 by ordering rule)
      d. ...
      e. Process N holds lock LN and waits for lock L1 (where LN < L1 for the circular wait to complete)
      f. Following this chain of dependencies: L1 < L2 < L3 < ... < LN < L1
   3. This creates a contradiction because it implies L1 < L1, which is impossible. A lock cannot have a lower precedence than itself.
   4. In order for Process N to create the circular dependency, it would hold the highest numbered lock LN but would need to request a lock with a lower number (L1), which directly violates the ordered locking protocol. This violation means our assumption of deadlock is false.
   5. The ordered locking mechanism prevents deadlocks because it ensures the wait-for graph remains acyclic - there can never be a cycle in the dependency chain when processes can only request higher-numbered locks than those they currently hold. PROOF FOUND.

6. **(1.5)** How can a deadlock occur with the dining philosophers? Would the ordered locking protocol work in this case?

Consider all philosophers (P) entering and sitting down at the same time. If their immediate response is to grab the fork to their right first, they will all be able to grab one fork. However, when they go to grab the left fork to begin eating, they are unable to as the P to their left has already grabbed that fork ('checked' it out). In this situation where all P's are at the table at the same time, we can enter a deadlock where all P's are waiting on each other to finish eating before they can receive the fork. However, they all will be continuously waiting since they all do not have the required number of forks to begin their process. Important to note that the "program" representing a philosopher says "obtain right, obtain left".

The ordered lock would work in this scenario so long as we appropriately label each of the locks associated with the forks (1 to N OR 0 to N-1). The philosophers would be required to pick up the lock with the lowest value first, and then the higher valued lock. When they release locks, it would be in reverse order with the higher valued lock being released first. This would effectively break the circular wait condition. You have not shown the break. This rule means that all but one will request forks in order (left, right). The other philosopher will request them in the order (right, left).

7. Let us consider the *producer-consumer* problem. Questions 7, 8, 9, 10 work their way to a practical solution. Here is the problem. One process, ***P***, produces and sends its output bytes to a buffer **B**, and a second process, ***C***, consumes bytes from buffer **B**. We want ***C*** to retrieve all the bytes from the buffer in exactly the same order as ***P*** placed them there, without duplicates or losses. For now, treat the buffer as infinite. There are two pointers, `in` and `out` (both initially 0); ***P*** places its next byte in `B[in]` and increments the pointer, `in++;` ***C*** retrieves its next byte from `B[out]` and increments the pointer, `out++`. Assume the buffer initially contains all 0.

   a. **(3.0)** Show an example in which the byte sequence seen by ***C*** is not the same as the sequence generated by ***P***. This error does not always happen; it is the result of a race condition.

      Consider process P (producer) accessing the buffer B at time T. The producer loads the current value of the in pointer (let's say 0) into its register to prepare for writing. Before the producer can write its data to B[0], process C (consumer) also loads the current value of the out pointer (also 0) into its register. The consumer then reads from B[0], which still contains uninitialized data (0) because the producer hasn't written yet. The consumer increments its out pointer to 1. Now the producer writes its data 'A' to B[0] and increments the in pointer to 1.

      For this scenario, here is what each process sees:

      Buffer after operations: [A, 0, 0, ...]

      Producer sequence: [A, ...]

      Consumer sequence: [0, ...]

      The issue here is that the consumer read uninitialized data instead of the actual data 'A' that the producer intended to send. This occurs

because the operations of writing data and incrementing the pointer are not atomic - they can be interrupted by the consumer's operations. The consumer has no way to know when it's safe to read, and the producer has no way to ensure its write completes before the consumer attempts to read from that location.

This demonstrates the race condition where the relative timing of producer and consumer operations determines whether data is correctly transferred, leading to lost or corrupted data transmission.

b. **(1.5)** Suppose that the OS could enforce the constraint `out<=in` by stopping ***C*** whenever the constraint is violated. Does that eliminate the race?

Not necessarily. For instance, consider the event in where the producer is in the process of writing data to the buffer. The producer writes at location B[in-1] and increments its pointer. At this point, B[in] is pointing to uninitialized space within the buffer. In addition, let us assume that B[in] = B[out] in this scenario. The consumer checks that the above condition is met and begins to read at B[out]. Since the consumer passed the above criteria, it is able to legally read at B[out] even though the producer has not written to that location yet.

This brings up the issue of atomicity. The consumer needs to be able to write to the buffer without the possibility of its action being interrupted by the producer. In the same manner, the producer needs to be able to read legal memory space only after it has been modified by the producer. For this to work, the critical section must be "write B[n]; in++"

Too complicated. If in=out, they can race accessing B[in].

8. Suppose that the OS maintains a counter, `filled`, indicating the number of items in the buffer ready for pickup.

a. **(3.0)** When should ***P*** increment the counter `filled`?

Process P should only increment the counter after it has finished writing to the memory location at B[in] and successfully advanced the in pointer. For instance, the current buffer is filled from indices 0-4 ([1, 1, 1, 1, 1,...]) which all have not been read by the consumer yet, meaning the filled counter is at 4. Process P begins writing to index 5 but does not increment the counter until after index 5 has been written to and it has incremented the in counter. This will show the consumer that there are 5 items ready for pickup.

b. **(3.0)** When should ***C*** decrement the counter `filled`?

Process C should decrement the counter filled only after it has read one (and only one) item from the buffer and successfully advances the out pointer. This will show that there is one fewer item to be ready for pickup and that the out pointer has been successfully incremented. Just as for 8.a, this section ideally needs to be implemented atomically (in a

critical section) to prevent other processes from observing partial updates.

c. **(3.0)** When should the OS stop ***P***?

With the assumption that the buffer is unbounded, the OS does not need to stop P for space constraints. With infinite capacity, there is no risk of overwriting unread data, so stopping P isn't required for correctness. The OS should stop P if process C has both shared resources checked out (accessing the buffer and modifying the filled counter). One other consideration for the system programmers to make is that they can also have the OS stop P if process C is lagging far behind in reading the data. For instance, they can create an upper limit for the filled counter to ensure that process P does not write to the buffer in an infinite manner while process C just spins and waits.

d. **(3.0)** When should the OS stop ***C***?

The OS should stop (suspend) process C if the filled counter is at 0 (empty). At this point, there is nothing to consume and continuing would allow process C to read uninitialized data. The OS should block process C until the filled counter is at least one. Even with an unbounded buffer, it makes no sense for the consumer to read beyond what has already been written.

9. The previous questions assume the buffer is unbounded. Suppose now it is bounded at length `N`. The `in` and `out` pointers can be recycled, for example `in=(in+1) mod N`.

a. **(3.0)** Why is the constraint `out<=in` now meaningless?

Considering the situation where the in pointer is near the end of the buffer. As soon as it gets to the last index of the buffer, it will increment and go back to 0 (modulus arithmetic). In this situation, even though there may be one index left for the consumer to read, it will never be able to satisfy the above condition of out <= in.

More critically, when out = in after wraparound, we cannot distinguish between two completely different situations: an empty buffer (where the consumer has caught up to the producer) versus a full buffer (where the producer has lapped the consumer completely). Both states satisfy out <= in but require opposite actions - in the empty case we should stop the consumer, but in the full case we should stop the producer. The constraint out <= in cannot differentiate between these critical buffer states, making it useless for proper buffer management decisions.

Now suppose the OS maintains *two* counters for the bounded buffer: `filled` is the number of items in the buffer ready for pickup (initially `0`), and `empty` is the number of empty slots in the buffer (initially `N`).

b. **(3.0)** When should ***P*** be stopped?

In addition to the same criteria listed in 8.c, process P also needs to be stopped when it has reached the end of the buffer. In other words, process P should be suspended when the empty counter has reached 0. This signifies to process P that there is no space left in the buffer, and that it needs to wait for process C to read items from the buffer so that it may begin overwriting items in the buffer for continued use.

c. **(3.0)** When should ***C*** be stopped?

In addition to the same criteria listed in 8.d, process C needs to be stopped when the filled counter has reached zero (filled == 0). It must not start reading again until the filled counter has been incremented to at least a value of 1.

d. **(1.0)** Why do we need two counters to control the producer-consumer interaction in a finite buffer?

We need the filled counter to show the consumer the number of items that are ready to be read. This counter is also useful for the producer as it can check to see how far behind the consumer is in reading from the buffer. We need the empty counter to show the producer how many slots are left in the buffer before it is out of space. After it has reached the end of the buffer, it now has to wait until the consumer has started reading (decrementing the filled counter). This way, it can cross reference with the out pointer and move its in pointer back to the beginning of the buffer. The empty counter will now increment back to the number of slots that are available (number of slots that either are uninitialized or have already been read by the consumer). Both of these counters work in tandem to create an efficient process for managing buffer space. Short answer: semaphores test for 0.

10. **(3.0)** Translate the logic of Question 9 into a semaphore implementation of producer and consumer interacting with the finite buffer. Use two semaphores: `filled` (initially `0`) and `empty` (initially `N`).

Semaphore: filled
This will start with a value of 0 and increment every time the producer writes data to the buffer that needs to be read by the consumer. The producer will first wait on the empty semaphore (WAIT(empty)) to block if there are no empty slots remaining. It then conducts its normal operations on the buffer (write item to buffer and increment filled counter). Finally, it will signal to the filled counter (SIGNAL(filled)) that there is an additional item to be read by the consumer.

For example:

```
WAIT(empty);          // block if no slots available
If (entered C):
        B[in] = item;          // write item
        In = (in + 1) % N;     // increment in pointer
```

```
        SIGNAL(filled);         // signal that one more item is available
LOOP BACK
```

Semaphore: empty

This will start with a value of N and decrement every time the producer writes something to the buffer (and while those items remain unread by the consumer). The consumer will first wait on the filled semaphore to block if the buffer is empty. It then conducts its normal operations on the buffer (read item from buffer and decrement filled counter). Finally, it will signal to the empty semaphore that an additional slot in the buffer is available for use (item has already been read and slot is ready to be overwritten and/or deleted).

For example:

```
WAIT(filled);           // block if buffer empty
If (entered C meaning buffer no longer empty):
        Item = B[out];          // read item
        Out = (out + 1) % N;    // increment out pointer
        SIGNAL(empty);          // signal one more empty slot available for
writing if necessary
LOOP BACK
```